AUTD3 触覚ボードキット
サンプル解説
Rust 編

版	日付	内容
1.00	2026/05/27	初版
1.10	2026/11/10	修正

修正履歴

版数	発行日	改版内容
1.00	2026/05/27	初版
1.10	2026/11/10	4.2.2 ビルド、他 各自にオフライン環境(キャッシュ優先)でビルドされる様に、 ビルド時のコマンドに「--offline」オプションを追加。

目次

1. 概要 2
2. ハードウェア接続 2
3. 起動方法 3
4. サンプルプログラム実行 4
4.1. サンプル一覧 4
4.2. ビルド及び実行方法 5
4.2.1. プロジェクト作成 5
4.2.2. ビルド 6
4.2.3. 実行 6
5. サンプルプログラム解説 7
5.1. 共通 7
5.2. 単一焦点サンプル(sample01_single.rs) 9
5.3. 複数焦点サンプル(sample02_multi.rs) 15
5.4. 焦点移動及び ON/OFF サンプル(sample03_move.rs) 18
5.5. ベッセルビームサンプル(sample04_bessel.rs) 22
5.6. STM サンプル(sample05_stm.rs) 26
6. その他 30
6.1. 独自プログラムの作成と実行 30
6.2. 実行時のエラーなど 31

1. 概要

本書は、AUTD3 触覚ボードキットに付属するする Rust 版サンプルについて、
実行方法、及び、コードの解説等を記載いたします。

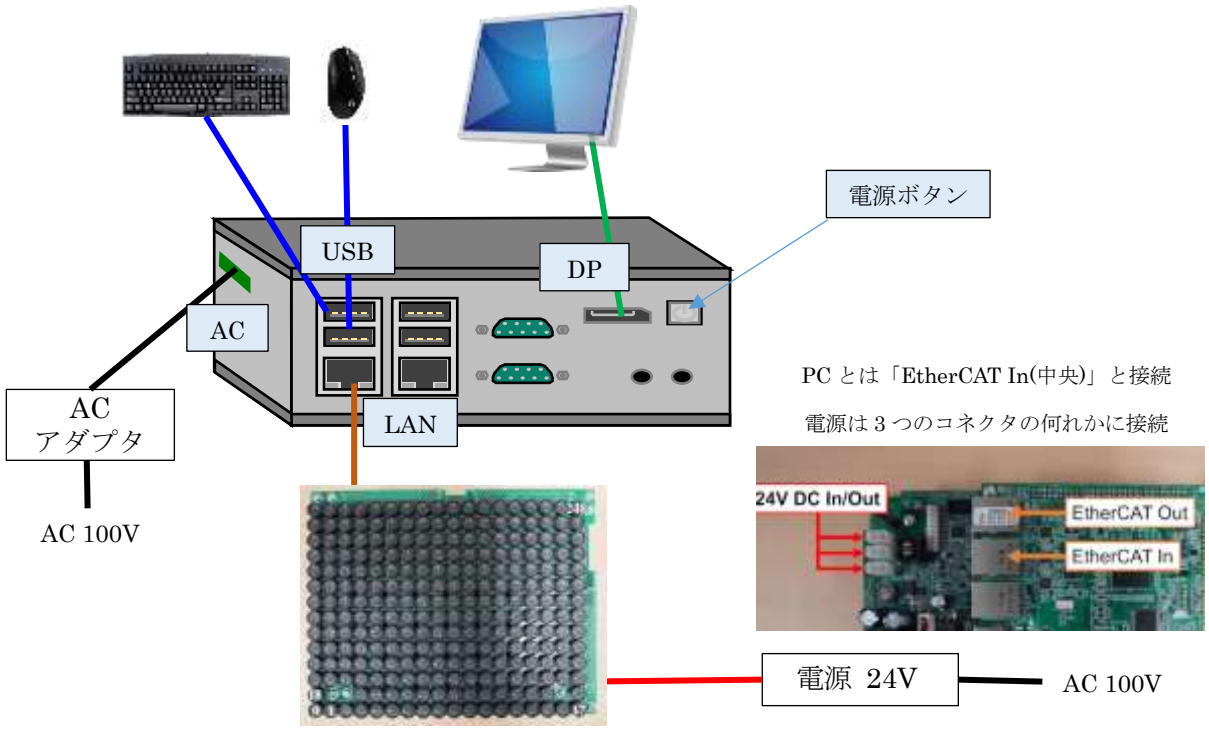
注意!

AUTD3 触覚ボードに目や耳を近づけないでください。

失明や耳を傷める恐れがあります。

2. ハードウェア接続

・ AUTD3 触覚ボードキットの各機器を、下記の様に接続します。(ADLink MEX-210)



・ キーボード/マウス/モニタはキットに付属しませんので別途ご用意願います。
なお PC 本体の画像出力は DP(DisplayPort)となります。必要に応じて DP-HDMI 変換機器等もご用意下さい。

・ PC 本体と AUTD3 ボードは、LAN ケーブル(カテゴリ 5e 以上)で接続します。
PC 側は LAN1(左側)に接続し、触覚ボード側は「中央のコネクタ」に接続します。
（上記画像の「EtherCAT in」に接続します）

3. 起動方法

・ PC 本体の電源ボタンを押して起動させます。

・ 起動後は下記の Ubuntu 画面がモニタに表示されます。



・ 出荷時はユーザログイン操作は省略させる設定となっています。
再ログイン等でユーザ/パスワードの入力を求められた場合は、下記を入力してください。

ユーザ名　：aiplay
パスワード：0

上記は出荷時の初期パスワードとなります。
必要に応じてユーザ側で変更してください。

4. サンプルプログラム実行

4.1. サンプル一覧

・ キットには下記の Rust サンプルプログラムが格納されています。

タイトル	ファイル名	概要
単一焦点	Sample01_single.rs	空間の一点に触覚点を生成する
複数焦点	Sample02_multi.rs	空間に二点の触覚点を生成する
焦点移動及び ON/OFF	Sample03_move.rs	焦点を移動、及び、ON/OFF の切り替えを行う
ベッセルビーム	Sample04_bessel.rs	ベッセルビームによる線状に触覚点を生成する
STM(単焦点)	Sample05_stm.rs	STM(時空間変調)により動きのある触覚点を生成する

・ サンプルプログラムはデスクトップの「RustSample」フォルダ下に格納されています。
フォルダパスは下記となります。

/home/aiplay/Desktop/RustSample

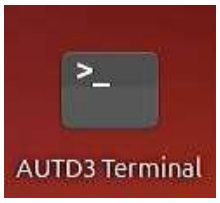
4.2. ビルド及び実行方法

Rust サンプルはソースファイルのみとなっているため、
実行するにはプロジェクトを作成しビルドを行う必要があります。

4.2.1. プロジェクト作成

・ ビルドを行うため、下記手順でサンプル用のプロジェクトを作成します。

① ターミナル起動(コンソール画面)
デスクトップで右クリックメニューより「端末で開く」を選択するか、
アイコン「AUTD3 Terminal」をダブルクリックしてターミナルウィンドウをを開きます。



アイコンをダブルクリック


「端末で開く」をクリック



② プロジェクト作成、及び、サンプルソースのコピー
下記コマンドにてプロジェクトフォルダを作成し、サンプルソースをコピーします。

```
$ cargo new --bin プロジェクトフォルダ名
$ cd プロジェクトフォルダ名
$ cp ~/Desktop/RustSample/サンプルソース名(Sample??_????.rs) src/main.rs
```

③ 外部ライブラリ(クレート)の追加
下記コマンドにてビルドに必要な外部ライブラリ(クレート)をプロジェクトに追加します。

```
$ cargo add autd3 --offline
$ cargo add autd3-link-ethercrab --offline
```

複数焦点サンプルの場合、下記も追加します

```
$ cargo add autd3-gain-holo --offline
```

必要な外部ライブラリはキット内にダウンロード済みですので、
「--offline」をつけてローカル(キャッシュ)にあるライブラリを追加します。
(キットはインターネットに接続しない環境での動作を想定していますので、
かならず「--offline」オプションを指定する様にして下さい)

4.2.2. ビルド

・ 下記コマンドにてビルドを行います。

```
$ cargo build --offline
```

オフライン環境でのビルドが行われる様に、
ビルド時も「--offline」オプションを指定を指定してください。

・ ビルドに成功した場合、下記に実行ファイルが作成されます。
プロジェクトフォルダ/target/debug/*プロジェクトフォルダ名*

4.2.3. 実行

・ AUTD3 デバイスとの通信で使用するアプリは、ネットワークインターフェースにて RAW パケットを使用する権限が必要となります。

・ この為、一時的に実行するには下記の様に sudo コマンドを用いて root 権限で実行します。

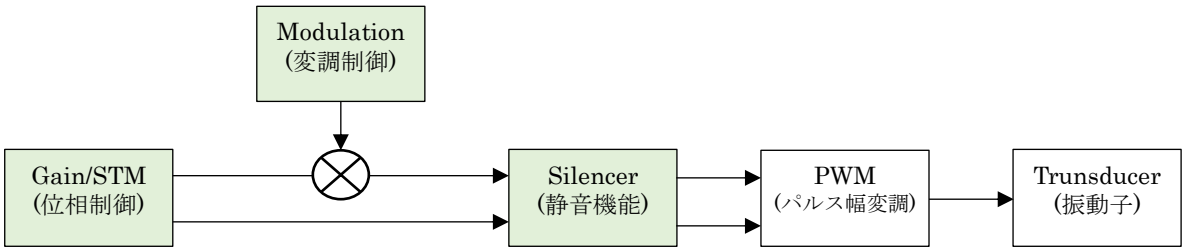
```
$ sudo ./target/debug/プロジェクトフォルダ名
```

5. サンプルプログラム解説

5.1. 共通

(1) AUTD3 触覚ボードの基本構造

・ AUTD3 触覚ボード内の動作概念図を下記に示します。



Gain	:	位相と振幅を生成します。
STM	:	時間に沿って変化する位相と振幅を生成します。
Modulation	:	AM 変調と加味します。
Silencer	:	消音化処理を行います。
PWM	:	各振動子を振動させる電気信号を生成します。
Trunsducer	:	電気信号を超音波に変換します。

位相制御にて位相と振幅を定義（触覚を発生させる位置等を決定）し、
変調制御にて AM 変調付加（触覚の感じを強くするなどの加工）を加え、
消音機能にて可聴音の除去（耳障りなノイズを抑制）させてから、
パルス幅変調で最終的な信号を生成し振動子（超音波スピーカー）を動作させます。

・ ユーザプログラムは緑色の部分に対してそれぞれの動作設定を指示することで、
様々な触覚を生む音場を発生させることができます。

Gain/STM 部	:	どの位置にどの様な焦点を発生させるかを指定します。
Modulation 部	:	触覚を決定する波を指定します。
Silencer 部	:	位相等が変化した場合に生じる可聴音を抑止する方法を指定します。

・ AUTD3 触覚ボードを動作させるための基本的な流れは下記の様になります。

① デバイス構成定義
② 通信方法決定
③ デバイス接続
④ 消音処理定義と送信
⑤ 位相制御の定義と送信
⑥ 変調制御の定義と送信
⑦ 位置や状態の変化が生じるまで待機（変更する場合は、④⑤⑥を再度行う）
⑧ デバイス切断

サンプルでは主に①～③、⑤～⑥をまとめて記述する場合があります。

(2) 通信方法

・ AUTD3 ライブラリでは複数の通信方法(Link)がサポートされています。

TwinCAT リンク

Beckhoff 社のソフトウェア製品を用いて通信を行います。(Windows 専用)

大量の AUTD3 ボードでも遅延を最小限にした高精度の通信を行えます。

EtherCrab リンク

Rust 言語で記述されたオープンソースなライブラリを使用して通信を行います。

数枚程度の AUTD3 ボード環境では最も安価で安定した通信を行えます。

Remote リンク

リモートサーバ経由で AUTD3 ボードと通信を行います。

プログラムの実行環境に通信用のライブラリ(TwinCAT 等)をインストールする必要がありません。

参考：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/link.html

・ キットに含まれるサンプルプログラムは Link(通信手段)として EtherCrab を使用しています。

・ 通信を開始する際に何れかのリンクを指定する必要がありますが、
基本的にそれ以降はリンク(通信方法)を気にすることなく、操作を行う事が可能です。
(EtherCrab を用いて動作確認後にリンク部分だけを TwinCAT 用に変更するだけで、
それ以外のコードは修正の必要がありません)

(3) 外部ライブラリ(クレート)の追加

・ AUTD3 ライブラリを使用する場合、下記の外部ライブラリ(クレート)をプロジェクトに追加する必要があります。

タイトル	ファイル名	追加が必要ライブラリ
単一焦点	sample01_single.rs	autd3, autd3-link-ethercrab
複数焦点	sample02_multi.rs	autd3, autd3-link-ethercrab, autd3-gain-holo
焦点移動及び ON/OFF	sample03_move.rs	autd3, autd3-link-ethercrab
ベッセルビーム	sample04_bessel.rs	autd3, autd3-link-ethercrab
STM(単焦点)	sample05_stm.rs	autd3, autd3-link-ethercrab

5.2. 単一焦点サンプル(sample01_single.rs)

(1) 動作概要

・ このサンプルは、１枚の AUTD3 デバイスの中央上部(デバイス表面から 15cm 上)に
触覚を感じる焦点を発生させます。



・ コンソールより下記コマンドを入力することでビルド・実行できます。

```
$ cargo new --bin sample01                          プロジェクト作成
$ cd sample01                                       プロジェクトフォルダに移動
$ cp ~/Desktop/RustSample/Sample01_single.rs src/main.rs   サンプルコードのコピー
$ cargo add autd3 --offline                         autd3 ライブラリ追加
$ cargo add autd3-link-ethercrab --offline          ethercrab ライブラリ追加
$ cargo build --offline                                   ビルド
$ sudo ./target/debug/sample01                      実行
```

・ Enter キーを押すと終了します。

(2) コード説明

・ コード全体を下記に示します。

```rust
// AUTD3 ライブラリの使用宣言
use autd3::prelude::*;

// EtherCrab ライブラリの使用宣言
use autd3_link_ethercrab::{EtherCrab, EtherCrabOption};

// ------------------------------------------------------------
// メイン関数
fn main() ->Result<(), Box<dyn std::error::Error>> {

    println!( "<単一焦点デモ>" );

    // デバイスへの接続
    println!( "デバイス接続" );
    let devices = [AUTD3 {
            pos: Point3::origin(),
            rot: UnitQuaternion::identity(),
        }];
    let link = EtherCrab::new(
            |idx, status| {
                eprintln!( "Device[{}]: {}", idx, status );
            },
            EtherCrabOption::default()
        );
    let mut autd = Controller::open( devices, link )?;

    // 可聴音を抑えるための指示
    autd.send( Silencer::default() )?;

    // 単一焦点の位相制御指示を生成
    let g = Focus {
                pos: autd.center() + Vector3::new( 0., 0., 150.0 * mm ),
                option: FocusOption::default()
            };

    // Sin 波形の変調制御指示を生成
    let m = Sine {
                freq: 150 * Hz,
                option: SineOption::default()
            };

    // 位相制御と変調制御を送信(動作開始)
    println!( "動作開始" );
    autd.send( (m, g) )?;

    // キー入力待ち(終了指示待ち)
    let mut _s = String::new();
    println!( "Enter キー入力で終了" );
    std::io::stdin().read_line( &mut _s )?;

    // デバイスとの接続終了
    autd.close()?;

    // 終了
    Ok( () )
}
```

使用するモジュール(クレート)の宣言

「1 枚ボード構成」のデバイスのリスト

「EtherCrab による接続」
EtherCrab でエラーが発生した場合に
呼び出される処理をクロージャで指定

「1 枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

単一焦点を生成する位相制御(Gain)を定義

正弦波による AM 変調を定義

位相制御(Gain)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

・ コードの詳細について説明します。

```rust
// AUTD3 ライブラリの使用宣言
use autd3::prelude::*;

// EtherCrab ライブラリの使用宣言
use autd3_link_ethercrab::{EtherCrab, EtherCrabOption};
```

上記では使用する外部ライブラリクラスを宣言しています。

autd3：AUTD3 ライブラリ

autd3-link-ethercrab：EtherCrab による通信ライブラリ

```rust
    // デバイスへの接続
    println!( "デバイス接続" );
    let devices = [AUTD3 {
            pos: Point3::origin(),
            rot: UnitQuaternion::identity(),
        }];
    let link = EtherCrab::new(
            |idx, status| {
                eprintln!( "Device[{}]: {}", idx, status );
            },
            EtherCrabOption::default()
        );
    let mut autd = Controller::open( devices, link )?;
```

デバイスへの接続は Controller クラスの open メソッドにより行います。
引数として、AUTD3 デバイスのリストと接続に使用する Link として EtherCrab を指定しています。

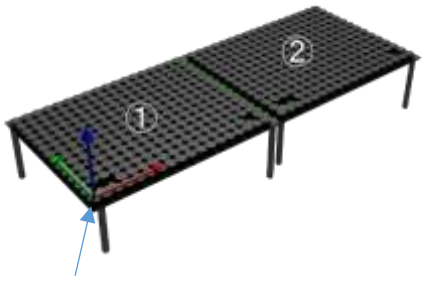
EtherCrab::new の第 1 引数には、通信でエラーが発生した場合の処理として、
エラー情報を出力するクロージャを指定しています。

本サンプルでは AUTD3 デバイスを１枚のみ使用しますのでリストは

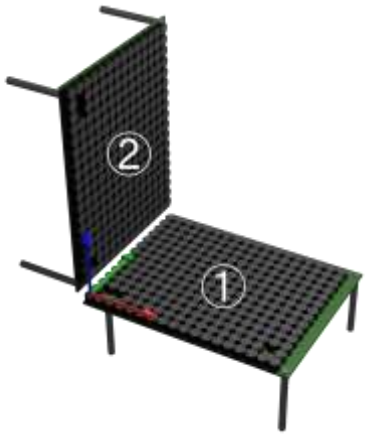
基準位置：(0, 0, 0)　　原点(Point3::origin)

向き　　：(1, 0, 0, 0)　　上向き(UnitQuaternion::identity)

の１件のみとなっていますが、複数の AUTD3 デバイスを使用する場合は、
各デバイス毎の位置と方向を列挙します。



サンプルでは上記の位置を原点(0,0,0)として、
接続台数によらず①の AUTD3 デバイスのみを使用する。



２枚の AUTD3 デバイスを使用する場合は、
位置を横にずらした②の情報をリストに追加する。

上記の様な構成の場合、
向きを 90 度傾けた AUTD3 デバイスを定義する。

AUTD3 デバイスの配置情報を指定すれば、ライブラリが個々のデバイス(振動子)の振動を自動的に計算する。
(位相制御や変調制御では基本的に AUTD3 デバイスの位置や向きなどを考慮する必要がない。
なおデバイス毎や振動子毎にマスクしたり個別に指定することもできる)

EtherCrab を用いて接続を行う場合は、
エラー発生時呼び出される処理(エラーハンドル)とオプションを指定する必要があります。

```
        |idx, status| {
            eprintln!( "Device[{}]: {}", idx, status );
        },
```

サンプルではエラーハンドルとしてエラーメッセージを表示するだけの処理をクロージャで指定しています。

またデフォルトのオプション(引数なし)を指定していますが、
必要によってネットワークインターフェースや同期タイミング等を指定できます。

```
// 可聴音を抑えるための指示
autd.send( Silencer::default() )?;
```

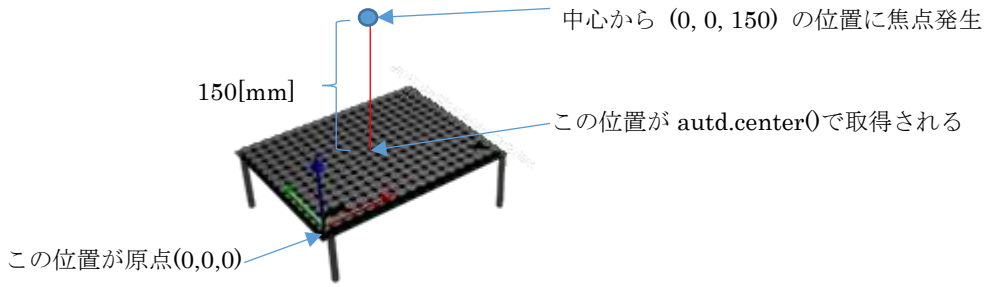
上記では、可聴音を抑制する Silencer を指示しています。
「Silencer::disable()」を送信すれば、消音処理を無効化できます。

```
// 単一焦点の位相制御指示を生成
let g = Focus {
            pos: autd.center() + Vector3::new( 0., 0., 150.0 * mm ),
            option: FocusOption::default()
        };

// Sin 波形の変調制御指示を生成
let m = Sine {
            freq: 150 * Hz,
            option: SineOption::default()
        };
```

上記で「単一焦点の位相制御 Focus」と「正弦波変調 Sine」の定義を行っています。

Focus の引数として「焦点座標(pos)」と「オプション(option)」を指定します。
「autd.center()」は全 AUTD3 デバイスの中心位置を表し、
オフセット成分「Vector3::new (0., 0., 150.0 * mm)」を加算することで
AUTD デバイス中心の上方向 150.0[mm]の位置を焦点座標としています。



変調制御として「正弦波:Sine」を指定しています。
変調なし(Static)の場合、動きがないと触覚を感じられません。
周波数は「150Hz」を指定していますが、この値を変更すると触感が変化します。

```rust
    // 位相制御と変調制御を送信(動作開始)
    println!( "動作開始" );
    autd.send( (m, g) )?;

    // キー入力待ち(終了指示待ち)
    let mut _s = String::new();
    println!( "Enter キー入力で終了" );
    std::io::stdin().read_line( &mut _s )?;
```

上記の「send()」にて位相制御と変調制御を AUTD3 デバイスに送信し、触覚を発生させます。

位相制御や変調制御は、送信する毎に上書きされます。(最後に送信された制御が使用されます)

Focus では単一位置で固定強度の焦点を発生させます。
細かく位相を変化させたい場合(焦点位置をブレさせたり、強さを変化させたい場合)などは
「STM：時空間変調」が有効です。

送信後は Enter キーが押されるまで待機します。
(プログラムが終了すると超音波出力も終了するため、
キー入力があるまで出力が継続される様にしています)

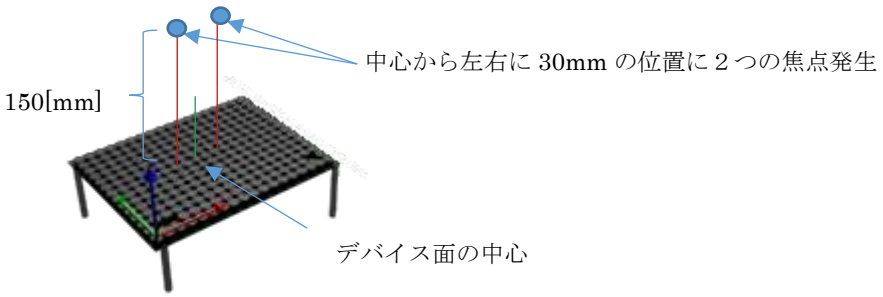
```rust
    // デバイスとの接続終了
    autd.close()?;
```

上記の「close()」により、AUTD デバイスとの接続を終了します。

5.3. 複数焦点サンプル(sample02_multi.rs)

(1) 動作概要

・ このサンプルは、１枚の AUTD3 デバイスの上に 2 つの触覚を感じる焦点を発生させます。



・ 多焦点生成アルゴリズムとして「GSPAT:Gershberg-Saxon for Phased Arrays of Transducers」を使用しています。この為、autd3-gain-holo ライブラリをプロジェクトに追加する必要があります。

・ コンソールより下記コマンドを入力することで実行できます。

```
$ cargo new --bin sample02                                  プロジェクト作成
$ cd sample02                                               プロジェクトフォルダに移動
$ cp ~/Desktop/RustSample/Sample02_multi.rs src/main.rs     サンプルコードのコピー
$ cargo add autd3 --offline                                 autd3 ライブラリ追加
$ cargo add autd3-link-ethercrab --offline                  ethercrab ライブラリ追加
$ cargo add autd3-gain-holo --offline                       gain-holo ライブラリ追加
$ cargo build --offline                                     ビルド
$ sudo ./target/debug/sample02                              実行
```

・ Enter キーを押すと終了します。

(2) コード説明

・ コード全体を下記に示します。

```rust
// AUTD3 ライブラリの使用宣言
use autd3::prelude::*;
use autd3_gain_holo::{ Pa, GSPAT, GSPATOption };

// EtherCrab ライブラリの使用宣言
use autd3_link_ethercrab::{EtherCrab, EtherCrabOption};

// ------------------------------------------------------------
// メイン関数
fn main() ->Result<(), Box<dyn std::error::Error>> {

    println!( "<複数焦点デモ>" );

    // デバイスへの接続
    println!( "デバイス接続" );
    let devices = [AUTD3 {
            pos: Point3::origin(),
            rot: UnitQuaternion::identity(),
        }];
    let link = EtherCrab::new(
            |idx, status| {
                eprintln!( "Device[{}]: {}", idx, status );
            },
            EtherCrabOption::default()
        );
    let mut autd = Controller::open( devices, link )?;

    // 可聴音を抑えるための指示
    autd.send( Silencer::default() )?;

    //
    let center = autd.center() + Vector3::new( 0., 0., 150.0 * mm );
    let focuslist = vec![
        (center - Vector3::new( 30.0 * mm, 0.0, 0.0 ), 5e3 * Pa),
        (center + Vector3::new( 30.0 * mm, 0.0, 0.0 ), 5e3 * Pa)
    ];
    let g = GSPAT::new (
        focuslist,
        GSPATOption::default()
    );

    // Sin 波形の変調制御指示を生成
    let m = Sine {
                freq: 150 * Hz,
                option: SineOption::default()
            };

    // 位相制御と変調制御を送信(動作開始)
    println!( "動作開始" );
    autd.send( (m, g) )?;

    // キー入力待ち(終了指示待ち)
    let mut _s = String::new();
    println!( "Enter キー入力で終了" );
    std::io::stdin().read_line( &mut _s )?;

    // デバイスとの接続終了
    autd.close()?;

    // 終了
    Ok( () )
}
```

使用するパッケージの宣言

「１枚ボード構成」のデバイスのリスト

「EtherCrab による接続」
EtherCrab でエラーが発生した場合に
呼び出される処理をクロージャで指定

「１枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

2 つの焦点座標のリストを定義
(center は二つの焦点の中心)

2 つの焦点を生成する位相制御(Gain)を定義

正弦波による AM 変調を定義

位相制御(Gain)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

・ コードの詳細について説明します。
なお可聴音を抑える為の指示部分までは、「単一焦点サンプル」とほぼ同様のため
「5.2 単一焦点サンプル」を参照願います。
ただし、下記のパッケージ宣言が追加されています。

```
use autd3_gain_holo::{ EmmisionConstraint, Pa, GSPAT, GSPATOption };
```

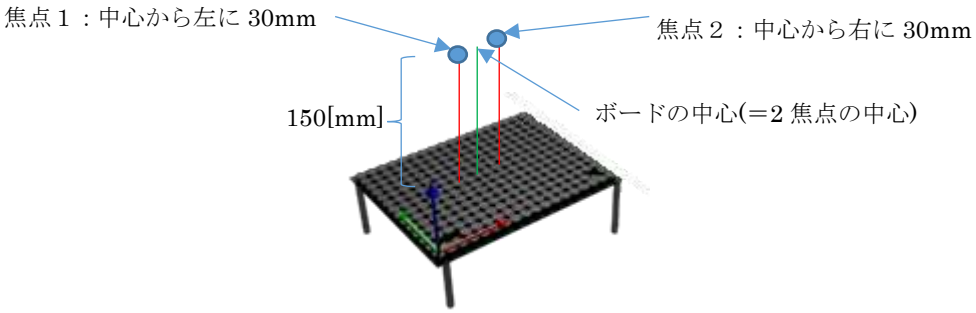
上記は、本サンプルで使用している多焦点生成アルゴリズム
「GSPAT:Gershberg-Saxon for Phased Arrays of Transducers」を使用するための宣言となります。

```
let center = autd.center() + Vector3::new( 0., 0., 150.0 * mm );
  let focuslist = vec![
      (center - Vector3::new( 30.0 * mm, 0.0, 0.0 ), 5e3 * Pa),
      (center + Vector3::new( 30.0 * mm, 0.0, 0.0 ), 5e3 * Pa)
  ];
```

上記で、GSPAT アルゴリズムによる 2 つの焦点を生成する位相制御を定義しています。

変数「center」には AUTD3 デバイスの中心から上 150mm の座標(＝2 焦点の中心)が格納されます。

GSPAT の引数は、「焦点座標リスト」と「オプション」を指定します。
「焦点座標リスト」は「焦点座標」と「圧力強度」からなるタプルのリストで、
上記コードの場合、
2 焦点の中心(center) から 左に 30mm の位置で、圧力 5×10^3[Pa]
2 焦点の中心(center) から 右に 30mm の位置で、圧力 5×10^3[Pa]
の 2 要素が指定されています。

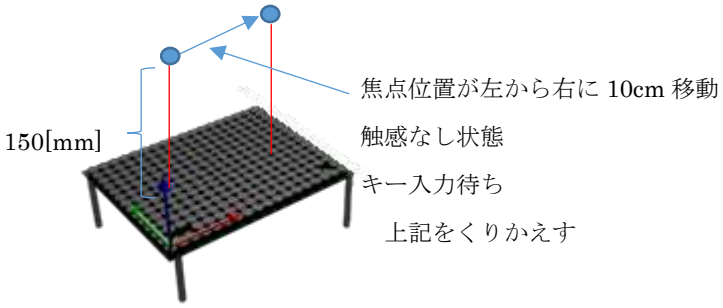


変調制御の定義(Sin 振幅)、及び、位相制御と変調制御の送信と待機部分についても
「単一焦点サンプル」と同様となります。

5.4. 焦点移動及び ON/OFF サンプル(sample03_move.rs)

(1) 動作概要

・ このサンプルは、AUTD3 デバイスの上の触覚を感じる焦点が左右に移動し、
右端に達した時に触感が一定期間(１秒)停止します。



・ 焦点の移動は、ミリ単位で位置をずらして単一焦点を発生させています。

・ 触感出力の停止は、Null という Gain を指示(送信)する事で実現します。

Focus 等の Gain を送信 : 触感 ON
Null を送信 : 触感 OFF

・ コンソールより下記コマンドを入力することで実行できます。

```
$ cargo new --bin sample03                          プロジェクト作成
$ cd sample03                                       プロジェクトフォルダに移動
$ cp ~/Desktop/RustSample/Sample03_move.rs src/main.rs   サンプルコードのコピー
$ cargo add autd3 --offline                         autd3 ライブラリ追加
$ cargo add autd3-link-ethercrab --offline          ethercrab ライブラリ追加
$ cargo build --offline                                   ビルド
$ sudo ./target/debug/sample03                      実行
```

・ 「空 Enter 入力で繰り返し(なにか入力して Enter で終了)」が表示された時、
何も入力せずに Enter キーを押すと繰り返し実行します。
何か入力して Enter キーを押すと、終了します。

(2) コード説明

・ コード全体を下記に示します。

```rust
// ライブラリの使用宣言
use autd3::prelude::*;
use autd3_link_ethercrab::{EtherCrab, EtherCrabOption};

// ------------------------------------------------------------
// メイン関数
fn main() ->Result<(), Box<dyn std::error::Error>> {

    println!( "<焦点移動デモ>" );

    // デバイスへの接続
    println!( "デバイス接続" );
    let devices = [AUTD3 {
            pos: Point3::origin(),
            rot: UnitQuaternion::identity(),
        }];
    let link = EtherCrab::new(
            |idx, status| {
                eprintln!( "Device[{}]: {}", idx, status );
            },
            EtherCrabOption::default()
        );
    let mut autd = Controller::open( devices, link )?;

    // 可聴音を抑えるための指示
    autd.send( Silencer::default() )?;

    // Sin 波形変調制御指示の指示(移動中は変更しないため事前に指示)
    let m = Sine {
                freq: 150 * Hz,
                option: SineOption::default()
            };
    autd.send( m )?;

    // 入力あるまでループ
    println!( "動作開始" );
    loop {

        // -50mm から+50mm まで移動
        for idx in -50..=50 {

            // 移動する位置に単一商店設定
            let g = Focus {
                pos: autd.center() + Vector3::new( idx as f32, 0.0, 150.0 * mm ),
                option: FocusOption::default()
            };
            autd.send( g )?;

            // 時間調整
            std::thread::sleep( std::time::Duration::from_millis( 10 ) );
        }

        // 停止
        autd.send( Null {} )?;

        // 終了確認
        let mut _s = String::new();
        println!( "空 Enter 入力で繰り返し(なにか入力して Enter で終了)" );
        std::io::stdin().read_line( &mut _s )?;
        if ! _s.trim().is_empty() {
            break;
        }
    }

    // デバイスとの接続終了
    autd.close()?;

    // 終了
    Ok( () )
}
```

使用するパッケージの宣言

「１枚ボード構成」のデバイスのリスト

「EtherCrab による接続」
EtherCrab でエラーが発生した場合に
呼び出される処理をクロージャで指定

「１枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

正弦波による AM 変調の定義と指示

移動ループ
（焦点位置を中心から-50mm～+50mm）

位相制御(Gain)の定義と送信

時間調整の為の待機

出力停止指示

終了確認

デバイスから切断

・ コードの詳細について説明します。

可聴音を抑える為の指示部分までと最後の切断処理は、「単一焦点サンプル」とほぼ同様のため
「5.2 単一焦点サンプル」を参照願います。

```
// Sin 波形変調制御指示の指示(移動中は変更しないため事前に指示)
let m = Sine {
            freq: 150 * Hz,
            option: SineOption::default()
        };
autd.send( m )?;
```

上記で正弦波による AM 変調を定義し、送信しています。
AM 変調自体は単焦点サンプルと同じですが、焦点が移動中は変化しないためループの前で送信しています。
（更新しない限り、最後に送信された変調が適用されます）

```
// 入力あるまでループ
println!( "動作開始" );
loop {

    // -50mm から+50mm まで移動
    for idx in -50..=50 {

        // 移動する位置に単一商店設定
        略

        // 時間調整
        略
    }

    // 停止
    autd.send( Null {} )?;

    // 終了確認
    略
}
```

上記は繰り返し動作（左から右への焦点移動、停止、終了確認）のループ処理になります。
（各項目のコードは次ページ以降で解説するため省略しています）

```
        // 移動する位置に単一商店設定
        let g = Focus {
            pos: autd.center() + Vector3::new( idx as f32, 0.0, 150.0 * mm ),
            option: FocusOption::default()
        };
        autd.send( g )?;
```

ループ内の単一焦点設定では、移動する焦点の位置に合わせた Gain(Focus)を生成し、送信しています。
autd.center()が全体の中心位置であり、-50～50 の範囲で変化する変数 idx を x 座標オフセットとして焦点位置 pos を設定しています。

```
    // 時間調整
    std::thread::sleep( std::time::Duration::from_millis( 10 ) );
```

上記の時間調整は、ゆっくりと焦点位置を移動させるための待機処理となります。

```
    // 停止
    autd.send( Null {} )?;
```

横移動ループが終了後、上記にて出力を停止しています。
ループして次の単一焦点設定を送信した時に出力は(新しい位置で)再開されます。

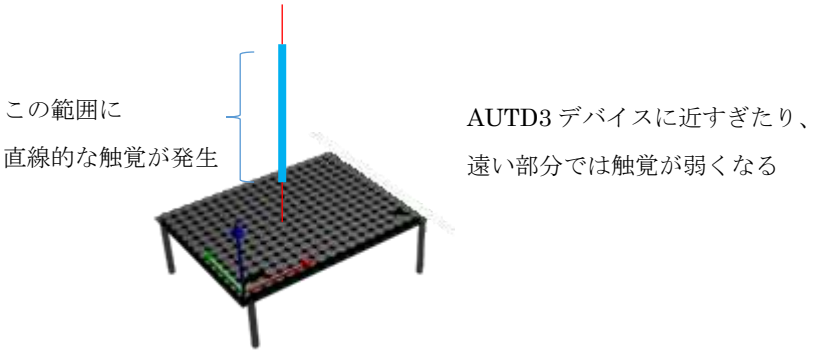
```
    // 終了確認
    let mut _s = String::new();
    println!( "空 Enter 入力で繰り返し(なにか入力して Enter で終了)" );
    std::io::stdin().read_line( &mut _s )?;
    if ! _s.trim().is_empty() {
        break;
    }
```

上記にて動作の終了確認を行っています。
コンソールから空入力以外がされた場合、ループを抜け終了します。

5.5. ベッセルビームサンプル(sample04_bessel.rs)

(1) 動作概要

・ このサンプルは、AUTD3 デバイスにてベッセルビームを発生させ、
一か所の焦点ではなく直線的な範囲の触覚を発生させます。



・ コンソールより下記コマンドを入力することで実行できます。

```
$ cargo new --bin sample04                                  プロジェクト作成
$ cd sample04                                               プロジェクトフォルダに移動
$ cp ~/Desktop/RustSample/Sample04_bessel.rs src/main.rs    サンプルコードのコピー
$ cargo add autd3 --offline                                 autd3 ライブラリ追加
$ cargo add autd3-link-ethercrab --offline                  ethercrab ライブラリ追加
$ cargo build --offline                                     ビルド
$ sudo ./target/debug/sample04                              実行
```

・ Enter キーを押すと、終了します。

(2) コード説明

・ コード全体を下記に示します。

```rust
// AUTD3 ライブラリの使用宣言
use autd3::prelude::*;

// EtherCrab ライブラリの使用宣言
use autd3_link_ethercrab::{EtherCrab, EtherCrabOption};

// ------------------------------------------------------------
// メイン関数
fn main() ->Result<(), Box<dyn std::error::Error>> {

    println!( "<ベッセルビームデモ>" );

    // デバイスへの接続
    println!( "デバイス接続" );
    let devices = [AUTD3 {
            pos: Point3::origin(),
            rot: UnitQuaternion::identity(),
        }];
    let link = EtherCrab::new(
            |idx, status| {
                eprintln!( "Device[{}]: {}", idx, status );
            },
            EtherCrabOption::default()
        );
    let mut autd = Controller::open( devices, link )?;

    // 可聴音を抑えるための指示
    autd.send( Silencer::default() )?;

    // ベッセルビームの位相制御指示を生成
    let g = Bessel {
                apex: autd.center(),
                dir: UnitVector3::new_normalize( Vector3::new(0.0, 0.0, 1.0) ),
                theta: 13.0 / 180.0 * std::f32::consts::PI * rad,
                option: BesselOption {
                    intensity: Intensity::MAX,
                    phase_offset: Phase::ZERO,
                }
            };

    // Sin 波形の変調制御指示を生成
    let m = Sine {
                freq: 150 * Hz,
                option: SineOption::default()
            };

    // 位相制御と変調制御を送信(動作開始)
    println!( "動作開始" );
    autd.send( (m, g) )?;

    // キー入力待ち(終了指示待ち)
    let mut _s = String::new();
    println!( "Enter キー入力で終了" );
    std::io::stdin().read_line( &mut _s )?;

    // デバイスとの接続終了
    autd.close()?;

    // 終了
    Ok( () )
}
```

使用するパッケージの宣言

「１枚ボード構成」のデバイスのリスト

「EtherCrab による接続」
EtherCrab でエラーが発生した場合に
呼び出される処理をクロージャで指定

「１枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

ベッセルビームを生成する
位相制御(Gain)を定義

正弦波による AM 変調を定義

位相制御(Gain)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

・ コードの詳細について説明します。

可聴音を抑える為の指示部分までと最後の待機/切断処理は、「単一焦点サンプル」とほぼ同様のため「5.2 単一焦点サンプル」を参照願います。

```
// ベッセルビームの位相制御指示を生成
let g = Bessel {
            apex: autd.center(),
            dir: UnitVector3::new_normalize( Vector3::new(0.0, 0.0, 1.0) ),
            theta: 13.0 / 180.0 * std::f32::consts::PI * rad,
            option: BesselOption {
                intensity: Intensity::MAX,
                phase_offset: Phase::ZERO,
            }

// Sin 波形の変調制御指示を生成
let m = Sine {
            freq: 150 * Hz,
            option: SineOption::default()
        };

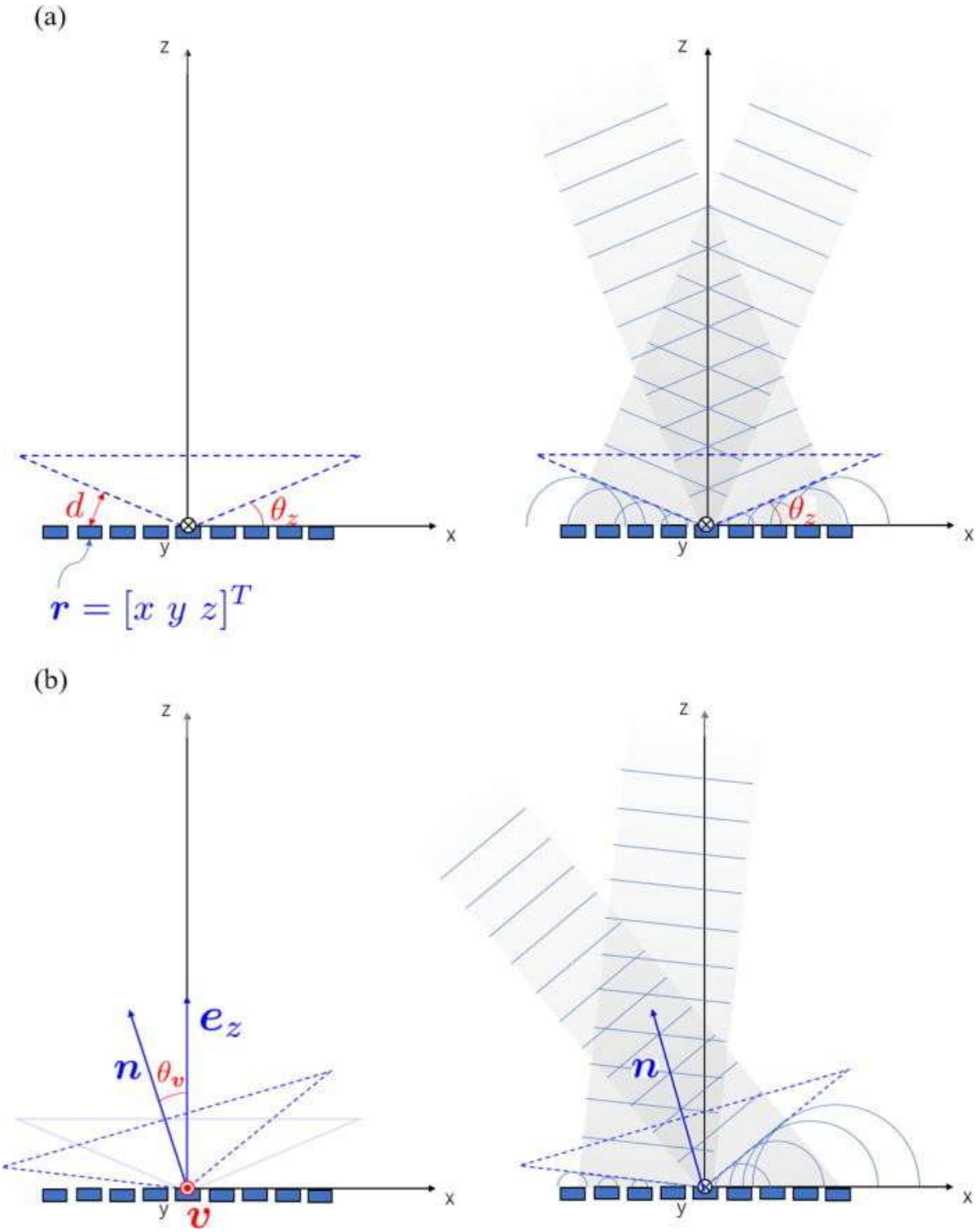
// 位相制御と変調制御を送信(動作開始)
println!( "動作開始" );
autd.send( (m, g) )?;
```

上記でベッセルビームを発生させる位相制御と AM 変調制御を定義し送信しています。

ベッセルビームの位相制御は、ビームを生成する仮想円錐の頂点(apex)とビームの方向(direction)、及び、ビームに垂直な面と仮想円錐側面との角度(theta)を指定して定義します。（次ページの図を参照）

上記の　Vector3::new(0.0, 0.0, 1.0)　部分の 3 つの数値(0.0 / 0.0 / 1.0)を変化させると、斜めに伸びる触覚範囲を生じさせることができます。

変調制御は単焦点サンプルと同じ「正弦波:Sine」を指定しています。

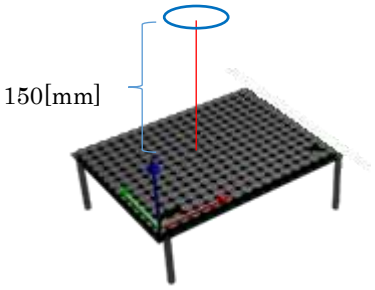


参考：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/gain/bessel.html

5.6. STM サンプル(sample05_stm.rs)

(1) 動作概要

・ このサンプルは、Spatio-Temporal Modulation:時空間変調をもちいて、
時間とともに変化する焦点を発生させます。



中心から上 150mm の位置を中心に
半径 30mm の円を描く様に触覚が発生
(1 秒周期で回転)

・ コンソールより下記コマンドを入力することで実行できます。

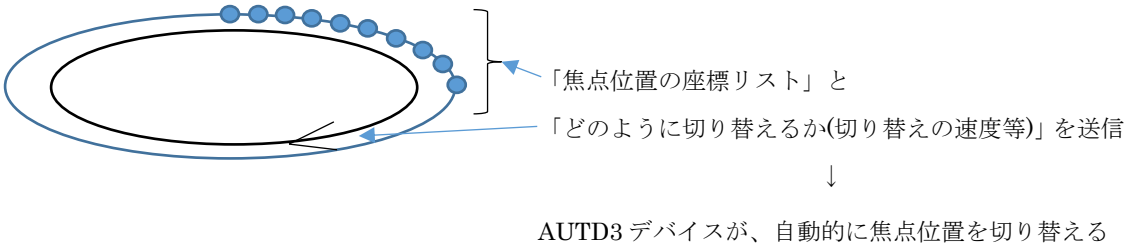
```
$ cargo new --bin sample05                          プロジェクト作成
$ cd sample05                                       プロジェクトフォルダに移動
$ cp ~/Desktop/RustSample/Sample05_stm.rs src/main.rs   サンプルコードのコピー
$ cargo add autd3 --offline                         autd3 ライブラリ追加
$ cargo add autd3-link-ethercrab --offline          ethercrab ライブラリ追加
$ cargo build --offline                                   ビルド
$ sudo ./target/debug/sample05                      実行
```

・ Enter キーを押すと、終了します。

・ サンプルは単焦点サンプルと同様に位相制御等を送信後はキー入力があるまでプログラムは待機状態となりますが、焦点移動サンプルの様に触覚の移動が繰り返されます。
(Linux 上のプログラムは待機状態となりますが、触覚ボード上で触覚箇所の動きを発生させます)

・ ライブラリでは下記の STM 機能を提供しています。
FociSTM ：８つまでの焦点を制御する STM
GainSTM ：任意の Gain を組み合わせて制御する STM
本サンプルでは FociSTM を使用します。

・ FociSTM では、「焦点の発生座標リスト」と「焦点の切り替え周期」を決定するパラメタを
AUTD3 デバイスに送信し、AUTD3 デバイス内で自動的に焦点座標を切り替えさせることで
焦点の動きを実現します。



本サンプルでは、「200 個の焦点座標リスト」と「1Hz」のパラメタを送信していますので、
円を描く 200 点の焦点が 1 秒周期で回転するような触覚が発生します。

(2) コード説明

・ コード全体を下記に示します。

```rust
// ライブラリの使用宣言
use autd3::prelude::*;
use autd3_link_ethercrab::{EtherCrab, EtherCrabOption};

// ----------------------------------------------------------
// メイン関数
fn main() ->Result<(), Box<dyn std::error::Error>> {

    println!( "<時空間変調デモ>" );

    // デバイスへの接続
    println!( "デバイス接続" );
    let devices = [AUTD3 {
            pos: Point3::origin(),
            rot: UnitQuaternion::identity(),
        }];
    let link = EtherCrab::new(
            |idx, status| {
                eprintln!( "Device[{}]: {}", idx, status );
            },
            EtherCrabOption::default()
        );
    let mut autd = Controller::open( devices, link )?;

    // 可聴音を抑えるための指示
    autd.send( Silencer::default() )?;

    // 動作指定
    let center = autd.center() + Vector3::new( 0.0, 0.0, 150.0 * mm );
    let radius = 30.0 * mm;
    let reso = 200;
    let stm = FociSTM {
        foci: (0..reso)
            .map( |i| {
                let theta = 2.0 * PI * i as f32 / reso as f32;
                let p = radius * Vector3::new( theta.cos(), theta.sin(), 0.0 );
                center + p
            })
            .collect::<Vec<_>>(),
        config: 1.0 * Hz
    };

    // 変調なしの変調制御指示を生成
    let m = Static { intensity: 0xFF };

    // 位相制御と変調制御を送信(動作開始)
    println!( "動作開始" );
    autd.send( (m, stm) )?;

    // キー入力待ち(終了指示待ち)
    let mut _s = String::new();
    println!( "Enter キー入力で終了" );
    std::io::stdin().read_line( &mut _s )?;

    // デバイスとの接続終了
    autd.close()?;

    // 終了
    Ok( () )
}
```

使用するパッケージの宣言

「1 枚ボード構成」のデバイスのリスト

「EtherCrab による接続」
EtherCrab でエラーが発生した場合に
呼び出される処理をクロージャで指定

「1 枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

焦点座標のリストと周期を指定して
FociSTM を定義

固定の変調制御を定義

位相制御(STM)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

・ コードの詳細について説明します。

可聴音を抑える為の指示部分までと最後の待機/切断処理は、「単一焦点サンプル」とほぼ同様のため「5.2 単一焦点サンプル」を参照願います。

```
    // 動作指定
    let center = autd.center() + Vector3::new( 0.0, 0.0, 150.0 * mm );
    let radius = 30.0 * mm;
    let reso = 200;
    let stm = FociSTM {
        foci: (0..reso)
            .map( |i| {
                let theta = 2.0 * PI * i as f32 / reso as f32;
                let p = radius * Vector3::new( theta.cos(), theta.sin(), 0.0 );
                center + p
            })
            .collect::<Vec<_>>(),
        config: 1.0 * Hz
    };
```

上記では中心位置(autd.center())を中心とした半径:radius(30.0)[mm]の円の円周上の
200 点の座標リスト(foci)と 1 Hz で繰り返す周期設定(config)を指定し、FociSTM(stm)を定義しています。

```
// 変調なしの変調制御指示を生成
    let m = Static { intensity: 0xFF };
```

本サンプルでは変調制御として Static を使用しています。

他サンプルの正弦波 Sine と違い AM 変調が付与されませんが、
STM により焦点が次々と変化するため触感を感じられます。
（正弦波 AM 変調を付与した場合より、より小さな触覚が感じられます）

6. その他

6.1. 独自プログラムの作成と実行

・ Ubuntu 標準のテキストエディタでサンプルコードの修正や独自プログラムの編集を行えます。
ソースファイルを編集/保存し、サンプルプログラムと同様のコマンドで
実行することができます。

新規にプログラムを生成し、実行するときの手順を下記に示します。

```
$ cargo new --bin プロジェクト名                    プロジェクト作成
$ cd プロジェクト名                                 プロジェクトフォルダに移動

    ~/Desktop/ロジェクト名/src/main.rs をエディタ等で編集

$ cargo add autd3 --offline                         autd3 ライブラリ追加
$ cargo add autd3-link-ethercrab --offline          ethercrab ライブラリ追加

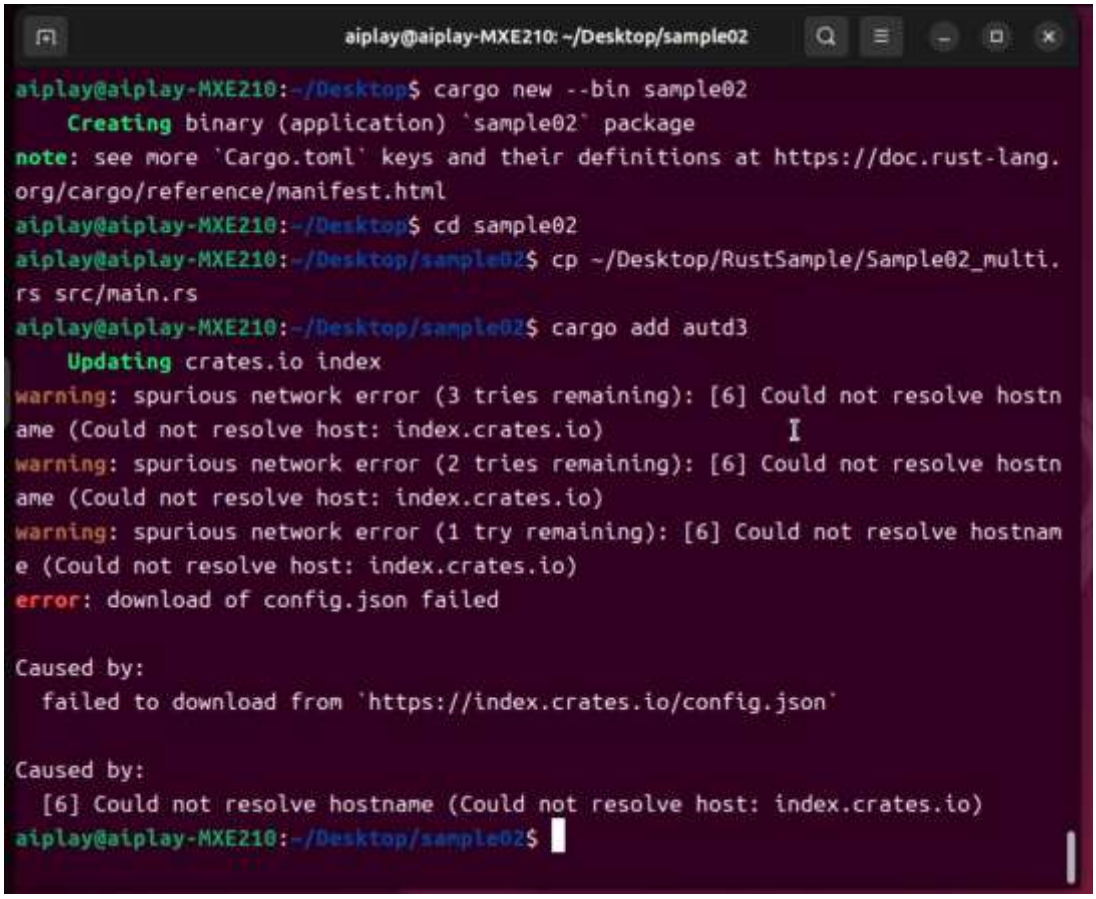
GSPAT 等の機能を使用する場合は、下記でライブラリを追加
$ cargo add autd3-gain-holo

$ cargo build --offline                                 ビルド
$ sudo ./target/debug/プロジェクト名                    実行
```

6.2. 実行時のエラーなど

(1) ライブラリ追加時/ビルド時にエラー

・ プロジェクトにライブラリを追加(cargo add)する際、下記の様な出力になった場合、
コマンドにオフライン指定(--offline)が抜けている可能性があります。



error: download of config.json failed

・ キットではインターネットに接続しない環境を想定しており、
必要なライブラリ等は事前にダウンロードされています。
ダウンロード済みのローカルにあるキャッシュを使用する場合、
下記の様に「--offline」オプションを指定してライブラリの追加を行ってください。

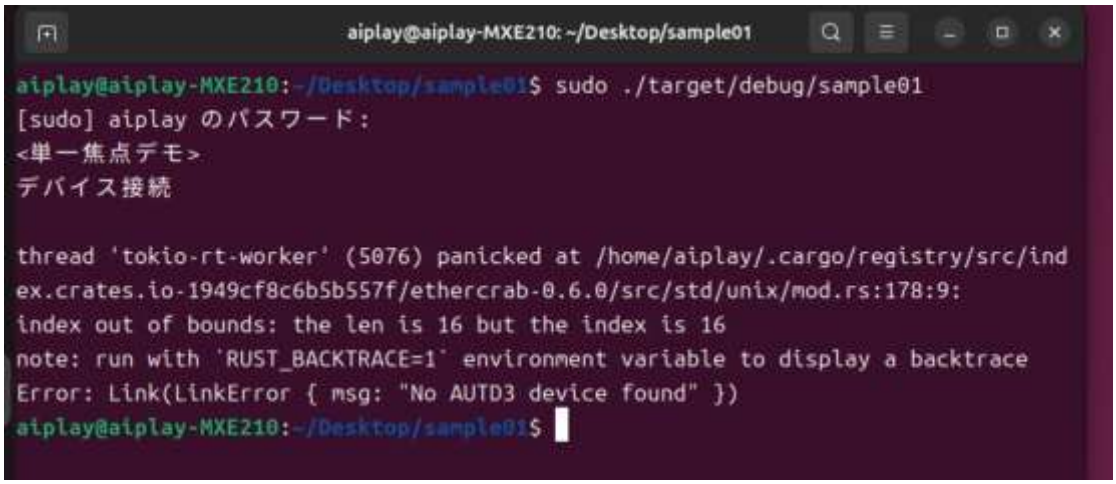
```
cargo add autd3 --offline
```

・ ビルドを行う際も、確実にダウンロード済みのローカルキャッシュが用いられる様に
「--offline」オプションを指定して下さい。

・ 独自のプログラムを作成する際に外部ライブラリを使用する場合は、
各ライブラリのローカル環境でのビルド方法を確認してください。

(2) 実行時に「NO AUTD3 device found」

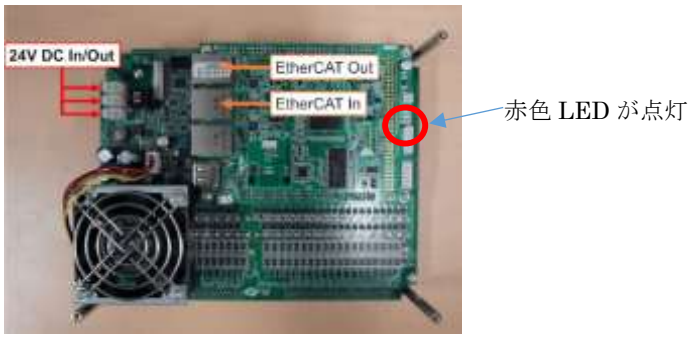
・ サンプル実行時に下記の様な出力となった場合、



Error : Link(LinkError { msg: "NO AUTD3 device found" })

下記の点を確認してください。

① AUTD3 デバイスの電源ケーブルが正しく接続されていることを確認してください。
また電源ユニット(24V)の電源が ON になっていることを確認してください。



AUTD3 デバイスは電源が入っている場合、上記の LED が点灯します。

② AUTD3 デバイスと PC が正しいコネクタに接続されていることを確認して下さい。
AUTD3 デバイス側は中央の LAN コネクタに接続します。
正しく接続されている場合、LAN コネクタの緑と黄色の LED が点灯します。
(PC とは HUB 等を経由せずに接続してください。また別のケーブルでの接続を確認下さい)

③ WiFi デバイスや USB-LAN デバイス(有線 LAN)等は接続しないでください。
複数のネットワークデバイスがあった場合でも、
通常はライブラリが自動的に適切なネットワークデバイスを選択しますが、
失敗した場合上記の様な状況になる場合があります。

(3) 実行時に AUTD3 デバイスのファンが動作

・ AUTD3 デバイスを長時間動作させボードに熱が生じた場合、自動的に冷却用のファンが動き出します。
ファンが動作している状態でも動作に問題はありません。

以上